

A Technical Introduction to Sequence to sequence learning

Section 1

This is the dot-attention mechanism. The particular version described in this section is "decoder cross-attention", as the output context vector is used by the decoder, and the input keys and values come from the encoder, but the query comes from the decoder, thus "cross-attention".

Section 2

As hand-crafting weights defeats the purpose of machine learning, the model must compute the attention weights on its own. Taking analogy from the language of database queries, we make the model construct a triple of vectors: key, query, and value. The rough idea is that we have a "database" in the form of a list of key-value pairs. The decoder sends in a query, and obtains a reply in the form of a weighted sum of the values, where the weight is proportional to how closely the query resembles each key. The decoder first processes the "<start>" input partially, to obtain an intermediate vector h_0^d , the 0th hidden vector of decoder. Then, the intermediate vector is transformed by a linear map W_Q into a query vector $q_0 = h_0^d W_Q$. Meanwhile, the hidden vectors outputted by the encoder are transformed by another linear map W_K into key vectors $k_0 = h_0^e W_K$, $k_1 = h_1^e W_K$, The linear maps are useful for providing the model with enough freedom to find the best way to represent the data. Now, the query and keys are compared by taking dot products: $q_0 k_0^T$, $q_0 k_1^T$, Ideally, the model should have learned to compute the keys and values, such that $q_0 k_0^T$ is large, $q_0 k_1^T$ is small, and the rest are very small. This can be interpreted as saying that the attention weight should be mostly applied to the 0th hidden vector of the encoder, a little to the 1st, and essentially none to the rest. In order to make a properly weighted sum, we need to transform this list of dot products into a probability distribution over $0, 1, \dots$. This can be accomplished by the softmax function, thus giving us the attention weights: $(w_{00}, w_{01}, \dots) = \text{softmax}(q_0 k_0^T, q_0 k_1^T, \dots)$. This is then used to compute the context vector: $c_0 = w_{00} v_0 + w_{01} v_1 + \dots$

Section 3

$(h_0, h_1, \dots, \text{"<start>"}) \rightarrow \text{"la"} (h_0, h_1, \dots, \text{"<start> la"}) \rightarrow \text{"la zone"} (h_0, h_1, \dots, \text{"<start> la zone"}) \rightarrow \text{"la zone de"} \dots (h_0, h_1, \dots, \text{"<start> la zone de contrôle international"}) \rightarrow \text{"la zone de contrôle international <end>"}$ Here, we use the special <start> token as a control character to delimit the start of input for the decoder. The decoding terminates as soon as "<end>" appears in the decoder output.

Section 4

seq2seq is an approach to machine translation (or more generally, sequence transduction) with roots in information theory, where communication is understood as an encode-transmit-decode process, and machine translation can be studied as a special case of communication. This viewpoint was elaborated, for example, in the noisy channel model of machine translation. In practice, seq2seq maps an input sequence into a real-numerical vector by using a neural network (the encoder), and then maps it back to an output sequence using another neural network (the decoder). The idea of encoder-decoder sequence transduction had been developed in the early 2010s. The papers most commonly cited as the originators that produced seq2seq are two papers from 2014. In the seq2seq as proposed by them, both the encoder and the decoder were LSTMs. This had the "bottleneck" problem, since the encoding vector has a fixed size, so for long input sequences, information would tend to be lost, as they are difficult to fit into the fixed-length encoding vector. The attention mechanism, proposed in 2014, resolved the bottleneck problem. They called their model RNNsearch, as it "emulates searching through a source sentence during decoding a translation". A problem with seq2seq models at this point was that recurrent neural networks are difficult to parallelize. The 2017 publication of Transformers resolved the problem by replacing the encoding RNN with self-attention Transformer blocks ("encoder blocks"), and the decoding RNN with cross-attention causally-masked Transformer blocks ("decoder blocks").

Section 5

The encoder is responsible for processing the input sequence and capturing its essential information, which is stored as the hidden state of the network and, in a model with attention mechanism, a context vector. The context vector is the weighted sum of the input hidden states and is generated for every time instance in the output sequences.

Section 6

Attention for seq2seq The attention mechanism is an enhancement introduced by Bahdanau et al. in 2014 to address limitations in the basic Seq2Seq architecture where a longer input sequence results in the hidden state

output of the encoder becoming irrelevant for the decoder. It enables the model to selectively focus on different parts of the input sequence during the decoding process. At each decoder step, an alignment model calculates the attention score using the current decoder state and all of the attention hidden vectors as input. An alignment model is another neural network model that is trained jointly with the seq2seq model used to calculate how well an input, represented by the hidden state, matches with the previous output, represented by attention hidden state. A softmax function is then applied to the attention score to get the attention weight. In some models, the encoder states are directly fed into an activation function, removing the need for alignment model. An activation function receives one decoder state and one encoder state and returns a scalar value of their relevance.